Tabla de contenidos

1. Nodo
2. Listas Ligadas
    a. `def agregar(valor)`
    b. `def obtener(posicion)`
    c. `def insertar(valor, posicion)`
    d. Iterar sobre Listas Ligadas
3. Otras estructuras de datos

Si bien Python, nos entregan una diversidad de estructuras de datos genéricas (listas, tuplas, *sets*, diccionarios, etc), la realidad es que en ciertas ocasiones estas estructuras no serán lo suficientemente complejas para reflejar la complejidad de lo que queremos representar.

Es por lo anterior que en este notebook estudiaremos las bases para representar agrupaciones de datos y las relaciones entre estos, utilizando OOP.

Nodo

El **nodo** corresponde a la base de las estructuras de datos. Un **nodo** es una unidad indivisible que contiene **datos** y que podemos identificar a través de un atributo llave (*key*). Cada nodo mantiene cero o más referencias con **otros nodos** a los que llamará **nodos vecinos** con los que mantiene una relación. Un conjunto de nodos, dado que se relacionan entre sí, nos permiten definir estructuras de datos más complejas y expresivas para representar información.

Listas Ligadas

Una lista ligada es una estructura que almacena nodos en un orden secuencial (como las listas, *stacks*, y colas), en que cada nodo posee una referencia a un único nodo **sucesor** (*next node*). El primer nodo de una lista ligada es denominado **cabeza** (*head*) de la lista, y el último, que no posee sucesor, es denominado **cola** (*tail*).

Notar que este concepto de **cola** se refiere a la denominación del último elemento de una lista (*tail*), y no a la estructura secuencial que, en inglés, denominamos *queue*. Para evitar confusiones cuando sea necesario, nos referiremos a él como **nodo cola**.

Para modelar un nodo de una lista ligada, utilizaremos una clase `Nodo` que tendrá como atributos un valor almacenado en el nodo (`valor`) y una referencia a su nodo sucesor (`siguiente`, también llamado `next`). Para el caso del nodo cola, su atributo `siguiente` será `None`, ya que es el último nodo en la secuencia.

```
from typing import Any

class Nodo:
    """
    Esta clase representa un nodo de una lista ligada
    """

    def __init__(self, valor: Any = None) -> None:
        """
        Inicializa la estructura del nodo
        """
        self.valor = valor
        self.siguiente = None  # Al crear un nodo,
                               # la referencia al siguiente nodo
comienza vacía

    def __repr__(self) -> str:
        return f"Nodo[{self.valor}]"
```

Creamos una clase abstracta `EstructuraSecuencialNodal` la cual almacenará `Nodos` y contedrá las características (atributos y metodos) que compartirán los distintos iterables que a lo largo de este notebook

```
from abc import ABC, abstractmethod
from typing import Any


class EstructuraSecuencialNodal(ABC):
    """
    Clase abstracta que representa una estructura secuencial
    en base a Nodos
    """

    # Todas nuestras estructuras secuenciales poseen:
    #   una cabeza y una cola que inicialmente estan vacías
    #   un largo
    #   un nodo inicial al cual referenciar
    @abstractmethod
    def __init__(self) -> None:
        self.cabeza = None
        self.largo = 0

    @abstractmethod
    def agregar(valor: Any) -> None:
        pass

    @abstractmethod
    def obtener(posicion: int) -> Any:
        pass
```

```
    @abstractmethod
    def insertar(valor: Any, posicion: int) -> None:
        pass

    def __len__(self) -> int:
        contador = 0
        nodo_actual = self.cabeza
        while nodo_actual is not None:
            nodo_actual = nodo_actual.siguiente
            contador += 1
        self.largo = contador
        return self.largo

# Creamos la base de nuestra Lista Ligada
class ListaLigada(EstructuraSecuencialNodal):
    """
    Clase que representa una lista ligada
    """

    def __init__(self) -> None:
        """
        Inicializa una lista ligada vacía,
        con una referencia nula a su cabeza y cola
        """
        super().__init__()
        # Como nuestra LL parte vacía, hacemos que la referencia
        # a su cola sea igual que a su cabeza
        self.cola = self.cabeza

    def agregar(valor: Any) -> None:
        return super().agregar(valor)

    def obtener(posicion: int) -> Any:
        return super().obtener(posicion)

    def insertar(valor: Any, posicion: int) -> None:
        return super().insertar(valor, posicion)

ll = ListaLigada()
len(ll)
```

```
0
```

Definiremos la estructura `ListaLigada`, usando una clase que referencia directamente, y solamente, a los nodos *cabeza* y *cola* de la secuencia. Para recorrer y alterar la lista ligada, es necesario acceder mediante las referencias de los nodos que la componen. Es decir, para acceder a un nodo que no es la cabeza o la cola de la lista, debemos utilizar las referencias de nodos desde la cabeza. La estructura que modelamos tendrá tres métodos:

1. `def agregar(valor)`

2. `def obtener(posicion)`
3. `def insertar(valor, posicion)`

A continuación, veamos cada método en detalle:

`def agregar(valor)`

Este método permite agregar un nuevo nodo al final de la lista ligada con el valor `valor`. Es similar al método `append` de `list`. Se puede implementar de manera muy eficiente con los siguientes pasos:

1. Crear un nodo nuevo, con el valor dado.
2. Al nodo **cola** actual, añadirle como sucesor el nodo recién creado.
3. Actualizar el atributo `cola` de la lista ligada, para que sea el nodo nuevo.

`def obtener(posicion)`

Este método permite recuperar el valor almacenado en el nodo que está en la posición `posicion` de la lista ligada. A diferencia de la estructura *list* de Python, no hay acceso indexado (`lista[indice]`), por lo que es necesario recorrer cada nodo hasta encontrar la posición deseada. Se comienza accediendo el nodo **cabeza** de la lista y mediante la referencia `siguiente` se accede al siguiente nodo en la lista. Se repite ese acceso hasta llegar a la posición `posicion` deseada.

`def insertar(valor, posicion)`

Este método permite agregar un nuevo nodo con valor `valor` en la posición `posicion` de la lista ligada. Esto se puede implementar de la siguiente manera:

1. Crear un nodo nuevo, con el valor dado.
2. Buscar el nodo de la posición en la que se quiere insertar (n_{aft}). Después de la inserción, este nodo quedará "después" del nodo nuevo.
3. Buscar el nodo anterior a la posición en que se quiere insertar (n_{pre}). Después de la inserción, este nodo quedará "antes" del nodo nuevo.
4. Hacer que el nodo siguiente del nodo nuevo sea n_{aft}.
5. Hacer que el nodo siguiente de n_{pre} sea el nodo nuevo.

Por ejemplo, en la siguiente animación se quiere insertar un nodo en la posición 2:

animación fabulosa de linkedlist
Fuente: VisuAlgo

Ahora, implementamos la lista ligada y cada una de las operaciones en el siguiente *snippet* de código:

```
from typing import Any


class ListaLigada(EstructuraSecuencialNodal):
    """
```

```
    Clase que representa una lista ligada
    """

    def __init__(self) -> None:
        """
        Inicializa una lista ligada vacía,
        con una referencia nula a su cabeza y cola
        """
        super().__init__()
        # Como nuestra LL parte vacía, hacemos que la referencia
        # a su cola sea igual que a su cabeza
        self.cola = self.cabeza

    def agregar(self, valor: Any) -> None:
        """
        Agrega un nodo al final de la cola, similar a lista.append
        """
        # Inicializamos el nuevo nodo
        nuevo = Nodo(valor)

        # Si la lista está vacía (no hay cabeza)
        if self.cabeza is None:
            # El nuevo nodo es la cabeza y cola de la lista
            self.cabeza = nuevo
            self.cola = self.cabeza
        else:
            # Agregamos el nuevo nodo como sucesor del nodo cola
actual
            self.cola.siguiente = nuevo
            # Actualizamos la referencia al nodo cola
            self.cola = self.cola.siguiente
        self.largo += 1

    def obtener(self, posicion: int) -> Any:
        """
        Busca el valor del nodo que está en la posición indicada,
        partiendo de 0
        """
        # Revisamos que la posición sea válida
        if posicion < 0:
            print(f'---No es posible obtener el valor en dicha
posición---')
            return None

        # Empezamos en la cabeza (nodo inicial en esta estructura)
        nodo_actual = self.cabeza

        # Recorremos secuencialmente la lista ligada siguiendo los
punteros
        # al nodo siguiente
```

```
        for _ in range(posicion):
            # Revisamos que no se haya llegado al final de la lista
            if nodo_actual is not None:
                nodo_actual = nodo_actual.siguiente

        # Si buscamos una posición mayor a la longitud de la lista
ligada
        if nodo_actual is None:
            return None  # Retorna "nada"
        return nodo_actual.valor

    def insertar(self, valor: Any, posicion: int) -> None:
        """
        Inserta un valor nuevo en una posición arbitraria
        """
        # Revisamos que la posición sea válida
        if posicion < 0:
            print(f'---No es posible insertar el valor en dicha
posición---')
            return None

        # Inicializamos el nuevo nodo
        nodo_nuevo = Nodo(valor)
        # Empezamos en la cabeza (nodo inicial en esta estructura)
        nodo_actual = self.cabeza

        # Caso particular: insertar en la cabeza
        if posicion == 0:
            # Actualizamos la cabeza
            nodo_nuevo.siguiente = self.cabeza
            self.cabeza = nodo_nuevo
            # Caso más particular. Si la lista estaba vacía,
            # actualizamos la cola
            if nodo_nuevo.siguiente is None:
                self.cola = nodo_nuevo
            # Terminamos de ejecutar la función
            self.largo += 1
            return

        # Buscamos el nodo predecesor
        for _ in range(posicion - 1):
            if nodo_actual is not None:
                nodo_actual = nodo_actual.siguiente

        # Si encontramos el predecesor, actualizamos las referencias
        if nodo_actual is not None:
            # Si no lo hacemos en este orden perdemos la referencia
            # al resto de la lista ligada
            nodo_nuevo.siguiente = nodo_actual.siguiente
            nodo_actual.siguiente = nodo_nuevo
```

```
            # Caso particular: si es que insertamos en la última
posición
            if nodo_nuevo.siguiente is None:
                self.cola = nodo_nuevo
            self.largo += 1
        else:
            print(f'---No es posible insertar el valor en dicha
posición---')
            return None

    def __repr__(self) -> str:
        """
        Forma una representación de la lista
        """
        string = ""
        nodo_actual = self.cabeza
        while nodo_actual is not None:
            string = f"{string}{nodo_actual.valor} → "
            nodo_actual = nodo_actual.siguiente
        return string
```

Acá probamos que nuestra lista ligada funciona:

```
ll = ListaLigada()
print(ll)
ll.agregar(2)
print(ll)
ll.agregar(3)
print(ll)
ll.agregar(5)
print(ll)
ll.agregar(7)
print(ll)
ll.agregar(11.0)
print(ll)
print(f'Largo de Lista Ligada: {len(ll)}')
```

```
2 →
2 → 3 →
2 → 3 → 5 →
2 → 3 → 5 → 7 →
2 → 3 → 5 → 7 → 11.0 →
Largo de Lista Ligada: 5
```

```
print(f'Largo de Lista Ligada: {ll.largo}')
print(f"Posición {0}: {ll.obtener(0)}")
print(f"Posición {2}: {ll.obtener(2)}")
print(f"Posición {4}: {ll.obtener(4)}")
print(f"Posición {20}: {ll.obtener(20)}")
```

```
Largo de Lista Ligada: 5
Posición 0: 2
Posición 2: 5
Posición 4: 11.0
Posición 20: None
```

```
ll.insertar("cuatro", 2)
print(ll)
print(f'\tLargo de Lista Ligada: {ll.largo}')
ll.insertar("cero", 0)
print(ll)
print(f'\tLargo de Lista Ligada: {ll.largo}')
ll.insertar("uno", 1)
print(ll)
print(f'\tLargo de Lista Ligada: {ll.largo}')
ll.insertar("trece", 8)
print(ll)
print(f'\tLargo de Lista Ligada: {ll.largo}')
ll.insertar("veinte", 20)
print(ll)
print(f'\tLargo de Lista Ligada: {ll.largo}')
```

```
2 → 3 → cuatro → 5 → 7 → 11.0 →
     Largo de Lista Ligada: 6
cero → 2 → 3 → cuatro → 5 → 7 → 11.0 →
     Largo de Lista Ligada: 7
cero → uno → 2 → 3 → cuatro → 5 → 7 → 11.0 →
     Largo de Lista Ligada: 8
cero → uno → 2 → 3 → cuatro → 5 → 7 → 11.0 → trece →
     Largo de Lista Ligada: 9
---No es posible insertar el valor en dicha posición---
cero → uno → 2 → 3 → cuatro → 5 → 7 → 11.0 → trece →
     Largo de Lista Ligada: 9
```

```
print(f'Largo de Lista Ligada: {ll.largo}')
print(f"Posición {0}: {ll.obtener(0)}")
print(f"Posición {7}: {ll.obtener(7)}")
print(f"Posición {100}: {ll.obtener(100)}")
```

```
Largo de Lista Ligada: 9
Posición 0: cero
Posición 7: 11.0
Posición 100: None
```

Iterar sobre Listas Ligadas

Como vimos anteriormente, si queremos recorrer los elementos de una lista ligada, debemos acceder de forma manual a cada uno de los nodos que la componen y acceder al nodo que lo sucede hasta llegar al nodo cola:

```
ll = ListaLigada()
ll.agregar(2)
ll.agregar(3)
ll.agregar(5)
ll.agregar(8)
ll.agregar(13)

nodo_actual = ll.cabeza

print(f'Recorreremos {len(ll)} elementos...')
while nodo_actual is not None:
    print(nodo_actual, end=" ")
    nodo_actual = nodo_actual.siguiente
```

```
Recorreremos 5 elementos...
Nodo[2] Nodo[3] Nodo[5] Nodo[8] Nodo[13]
```

Pero, aplicando nuestros conocimientos sobre iterables e iteradores, podemos adaptar esta estructura para que pueda ser recorrida mediante el uso de `for`. Para lograr lo anterior, deberemos implementar las clases iterador e iterable, junto con los métodos `__iter__` y `__next__`:

```
from typing import Any
from __future__ import annotations


class IteradorListaLigada:
    def __init__(self, cabeza: Nodo) -> None:
        self.nodo_actual = cabeza

    def __iter__(self) -> IteradorListaLigada:
        return self

    def __next__(self) -> Any:
        if self.nodo_actual is None:
            raise StopIteration("Llegamos al final")
        else:
            nodo = self.nodo_actual
            self.nodo_actual = self.nodo_actual.siguiente
            return nodo

class IterableListaLigada(ListaLigada):

    def __iter__(self) -> IteradorListaLigada:
        return IteradorListaLigada(self.cabeza)

lista_iterable = IterableListaLigada()
lista_iterable.agregar(2)
lista_iterable.agregar(3)
lista_iterable.agregar(5)
```

```
lista_iterable.agregar(8)
lista_iterable.agregar(13)

print(lista_iterable)
```

```
2 → 3 → 5 → 8 → 13 →
```

```
print(f'Recorreremos {len(lista_iterable)} elementos...')
for nodo in lista_iterable:
    print(nodo, end=' ')
```

```
Recorreremos 5 elementos...
Nodo[2] Nodo[3] Nodo[5] Nodo[8] Nodo[13]
```

Otras estructuras de datos

Además de las Listas Ligadas, existen múltiples estructuras de datos basadas en nodos:

- Árboles
  - Árboles binarios
  - Árbol de Búsqueda Binaria
  - Árbol *Trie*
  - *Heap*
- Grafos
  - Grafos direccionales
  - Grafos bidireccionales
- Entre otros

Debido al alcance de este curso, no podremos profundizar tanto en este contenido. Si estás interesado en aprender más, te invitamos a investigar sobre el curso "IIC2133 - Estructuras de Datos y Algoritmos".